Fundamentals of Data Science
21CSS202T

Unit V

VISUALIZATION 9 hours

Advantages and usecases, working with Matplotlib to plot different visuals, Working with Seaborn to plot different visuals, Univariate graphs for numeric and categorical data, Bivariate graphs for numeric and categorical data, Multivariate Graphs, Choosing appropriate graphical techniques, using graph to explore the data insights, Introduction to dashboards.

T13: Building programs to visualize the dataframe in matplotlib and seaborn

T14: Building programs to visualize the univariate, bivariate and multivariate relation

T15: Case study with all the appropriate graphs to visualize the relationship in the data

Data Visualization

- Data visualization is the most important step in the life cycle of data science, data analytics, or we can say in data engineering. It is more impressive, interesting and understanding when we represent our study or analysis with the help of colours and graphics.
- Using visualization elements like graphs, charts, maps, etc., it becomes easier for clients to understand the underlying structure, trends, patterns and relationships among variables within the dataset.
- Simply explaining the data summary and analysis using plain numbers becomes complicated for both, people coming from technical and non-technical backgrounds.
- Data visualization gives us a clear idea of what the data wants to convey to us. It makes data neutral for us to understand the data insights.

Data Visualization

Data Visualization

- Data visualization involves operating a huge amount of data and converts it into meaningful and knowledgeable visuals using various tools.
- For visualizing data we need the best software tools to handle various types of data in structured or unstructured format from different sources such as files, web API, databases, and many more.

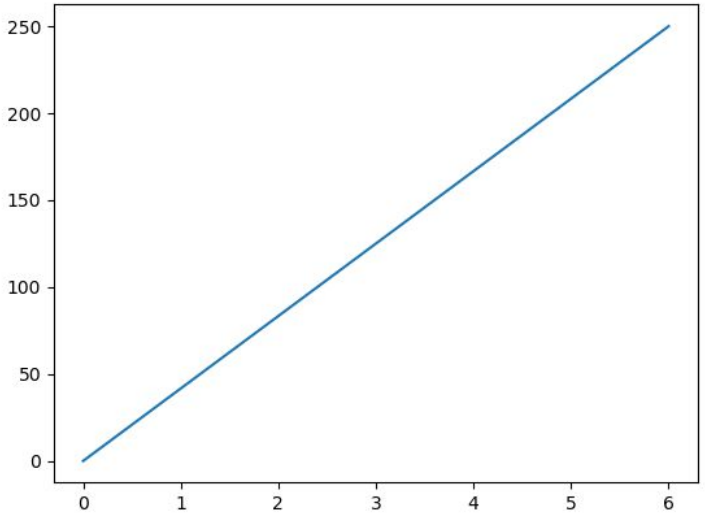
Data Visualization

Pyplot

- Draw a line in a diagram from position (0,0) to position (6,250):
- import matplotlib.pyplot as plt
  import numpy as np

  xpoints = np.array([0, 6])
  ypoints = np.array([0, 250])

  plt.plot(xpoints, ypoints)
  plt.show()



Pyplot

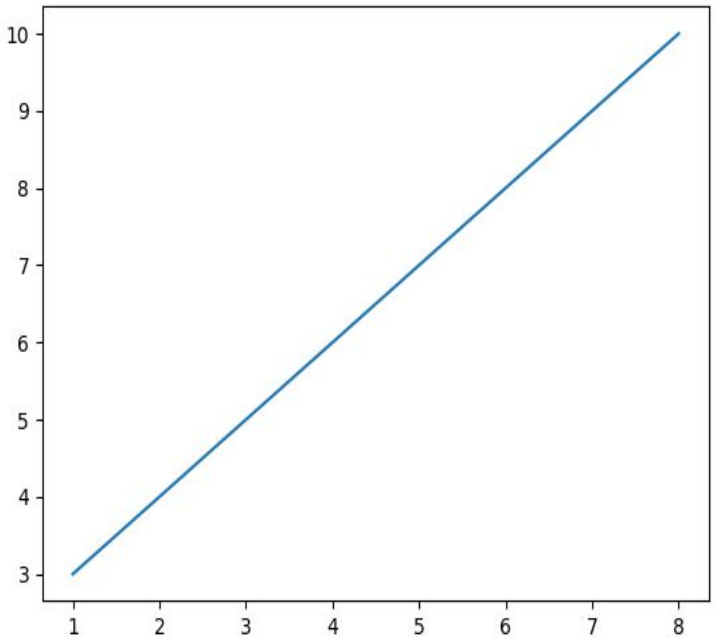
- The plot() function is used to draw points (markers) in a diagram.
- By default, the plot() function draws a line from point to point.
- The function takes parameters for specifying points in the diagram.
- Parameter 1 is an array containing the points on the **x-axis**.
- Parameter 2 is an array containing the points on the **y-axis**.
- If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

Pyplot

- Draw a line in a diagram from position (1, 3) to position (8, 10):
- import matplotlib.pyplot as plt
  import numpy as np

  xpoints = np.array([1, 8])
  ypoints = np.array([3, 10])

  plt.plot(xpoints, ypoints)
  plt.show()

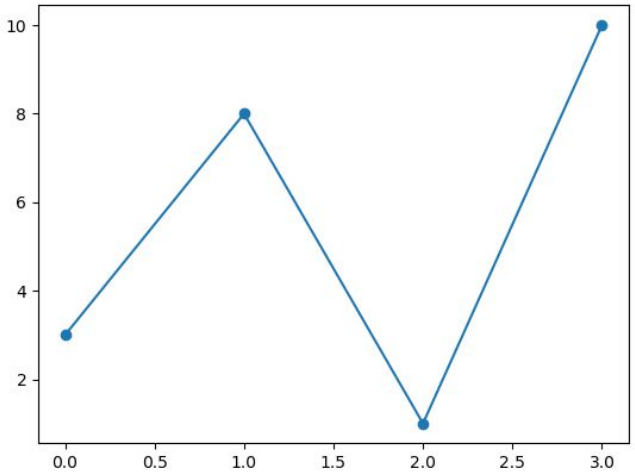


Marker

- You can use the keyword argument **marker** to emphasize each point with a specified marker.
- import matplotlib.pyplot as plt
  import numpy as np

  ypoints = np.array([3, 8, 1, 10])

  plt.plot(ypoints, marker = 'o')
  plt.show()

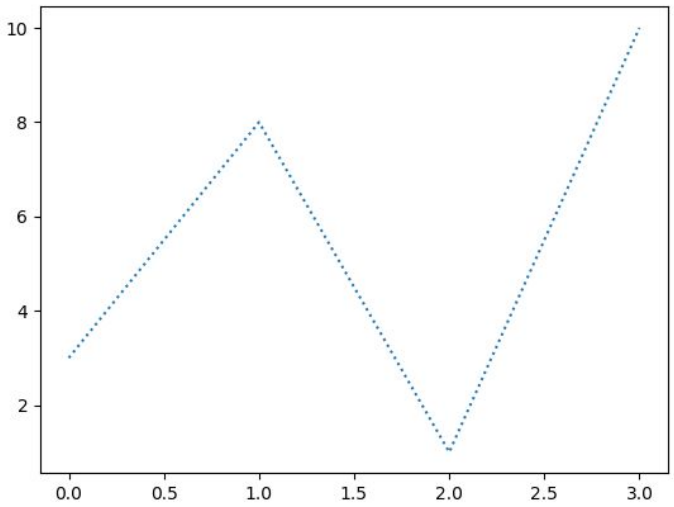


Linestyle

- You can use the keyword argument **linestyle**, or shorter **ls**, to change the style of the plotted line.
- import matplotlib.pyplot as plt
  import numpy as np

  ypoints = np.array([3, 8, 1, 10])

  plt.plot(ypoints, linestyle = 'dotted')
  plt.show()



Labels

- With Pyplot, you can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

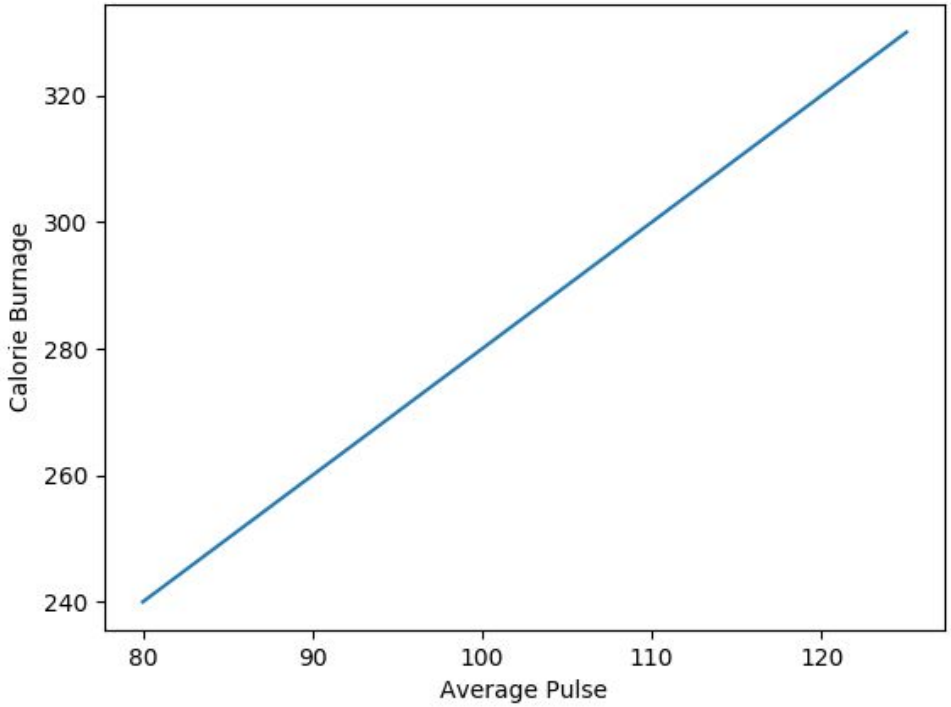
plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

Labels

- With Pyplot, you can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.



Title

- With Pyplot, you can use the **title()** function to set a title for the plot.
- import numpy as np
  import matplotlib.pyplot as plt

  x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
  y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

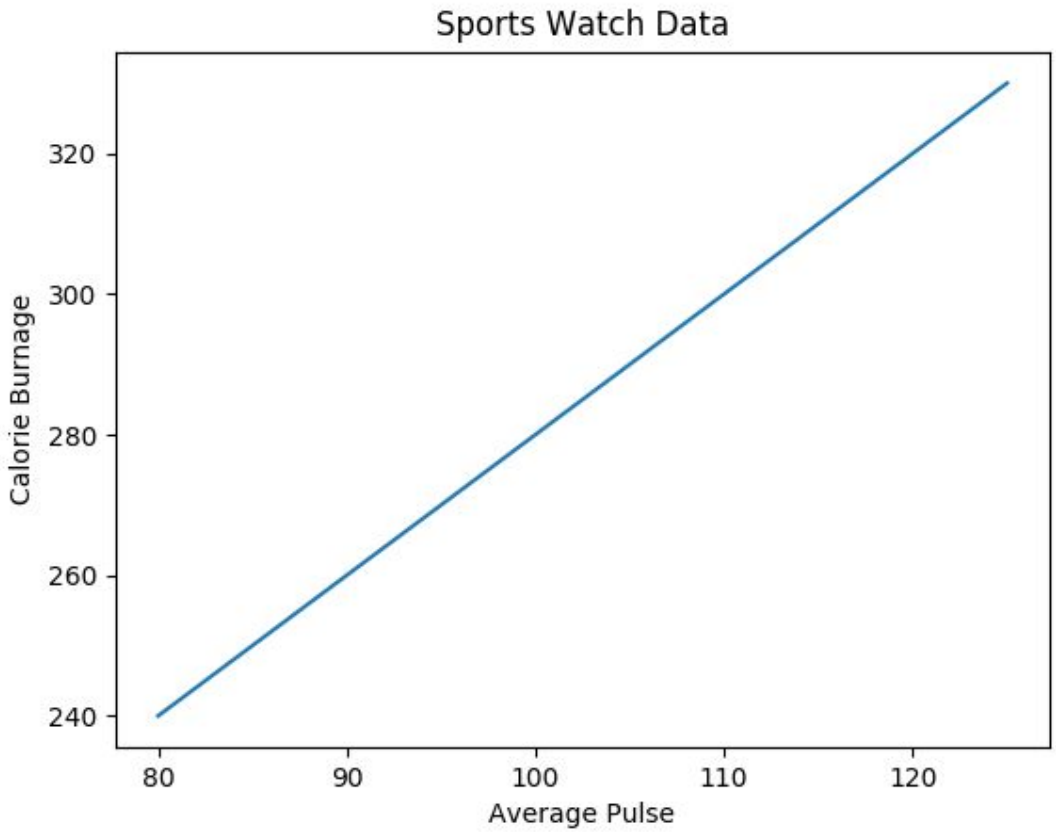
  plt.plot(x, y)

  plt.title("Sports Watch Data")
  plt.xlabel("Average Pulse")
  plt.ylabel("Calorie Burnage")

  plt.show()

Title

- With Pyplot, you can use the **title()** function to set a title for the plot.



Grid Lines

- With Pyplot, you can use the **grid()** function to add grid lines to the plot.

- ```
  import numpy as np
  import matplotlib.pyplot as plt

  x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
  y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

  plt.title("Sports Watch Data")
  plt.xlabel("Average Pulse")
  plt.ylabel("Calorie Burnage")

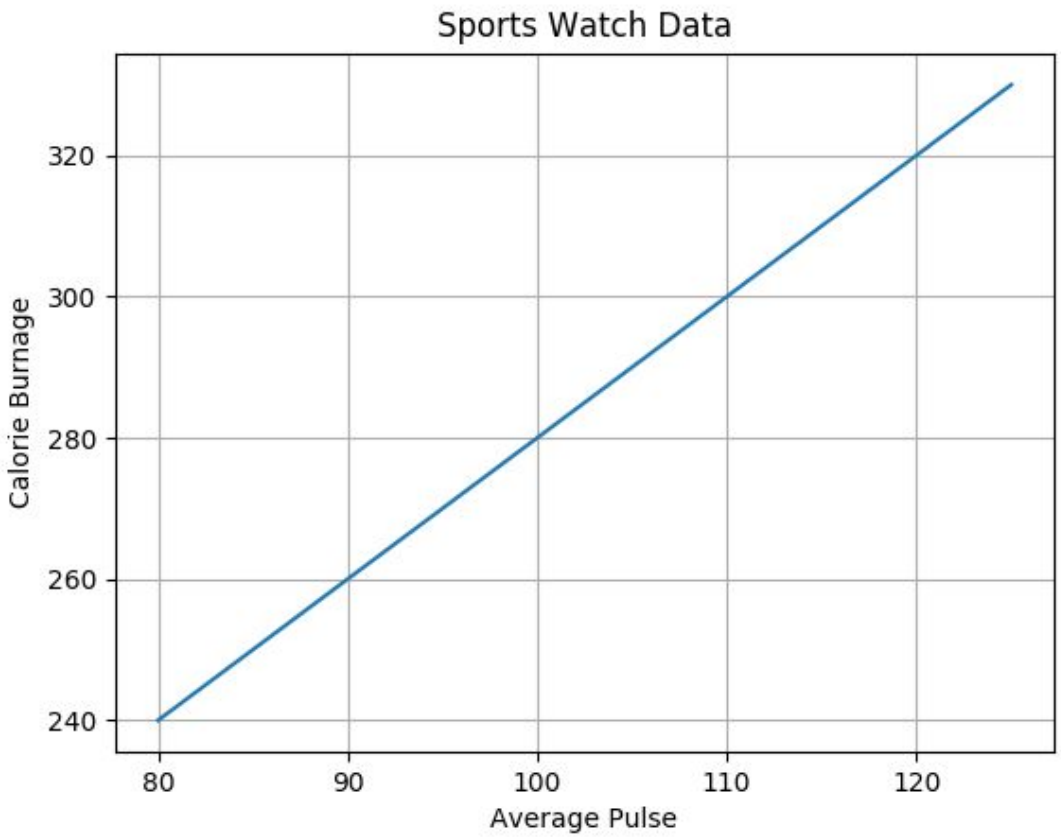
  plt.plot(x, y)

  plt.grid()

  plt.show()
  ```
  ```

Grid Lines

- With Pyplot, you can use the **grid()** function to add grid lines to the plot.



subplots

- With the subplot() function you can draw multiple plots in one figure:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

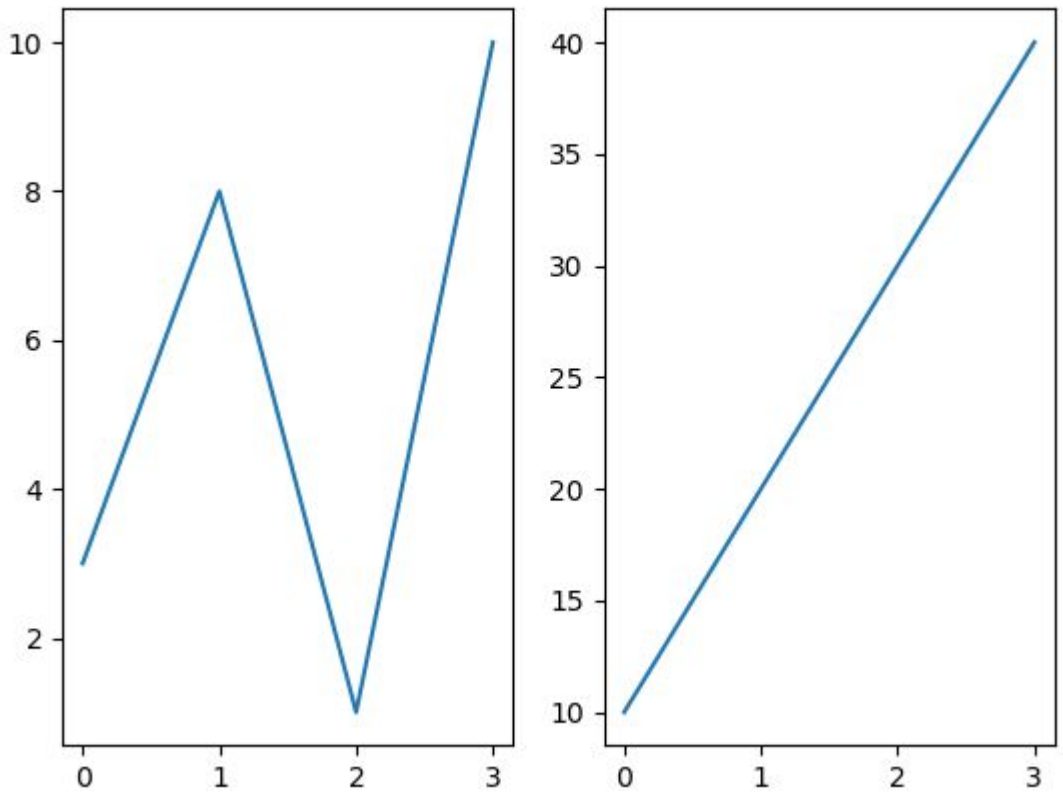
plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```

subplots



subplots

- The subplot() function takes three arguments that describes the layout of the figure.
- The layout is organized in rows and columns, which are represented by the *first* and *second* argument.
- The third argument represents the index of the current plot.
- plt.subplot(1, 2, 1)
  #the figure has 1 row, 2 columns, and this plot is the *first* plot.
-
  plt.subplot(1, 2, 2)
  #the figure has 1 row, 2 columns, and this plot is the *second* plot.

subplots

- Draw 2 plots on top of each other:
- import matplotlib.pyplot as plt
  import numpy as np

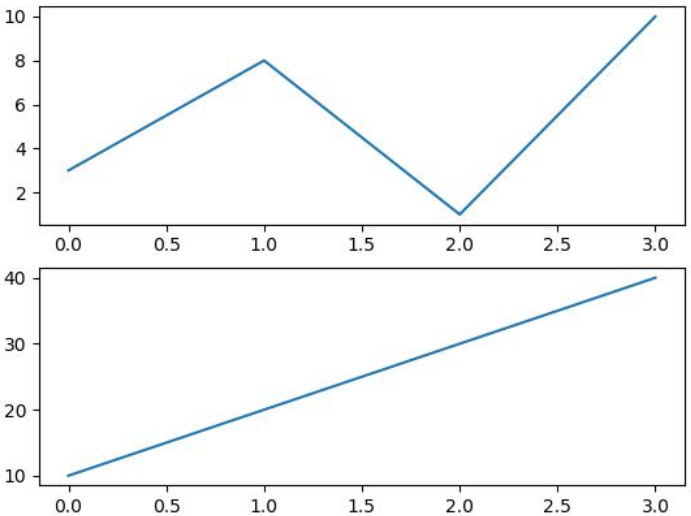
  #plot 1:
  x = np.array([0, 1, 2, 3])
  y = np.array([3, 8, 1, 10])

  plt.subplot(2, 1, 1)
  plt.plot(x,y)

  #plot 2:
  x = np.array([0, 1, 2, 3])
  y = np.array([10, 20, 30, 40])

  plt.subplot(2, 1, 2)
  plt.plot(x,y)

  plt.show()



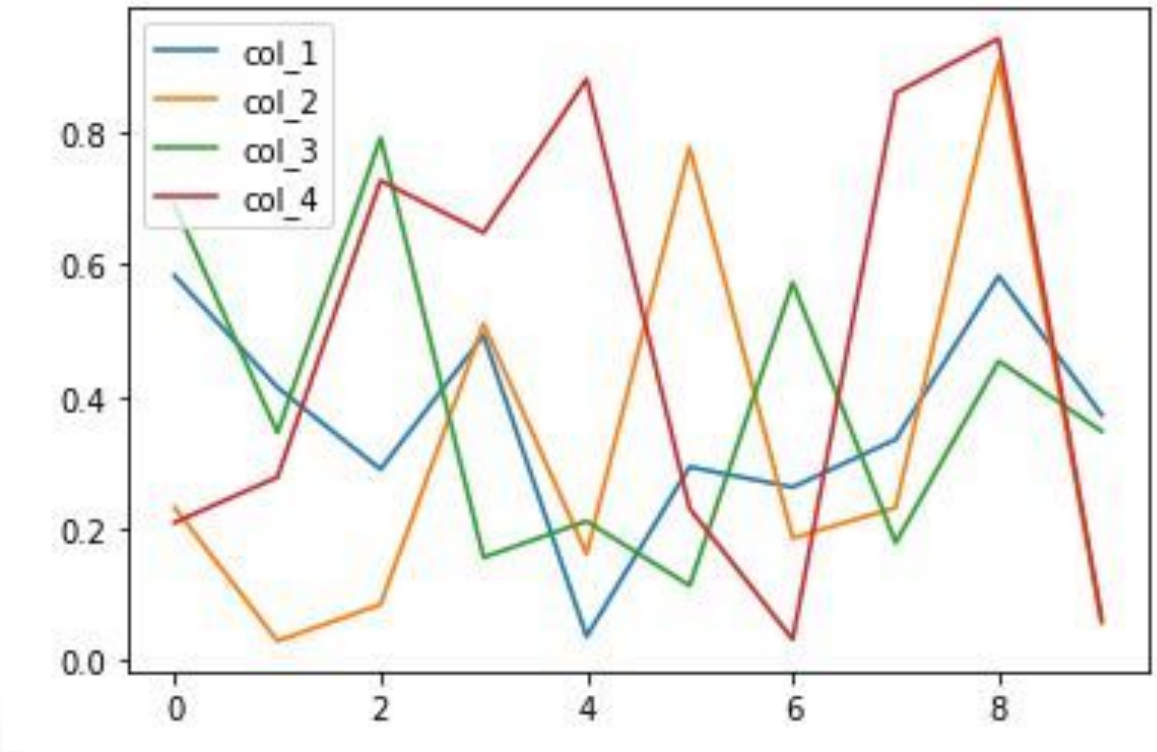
Types of plots

- Line plot
- Bar plot
- Histogram
- Piechart
- Scatterplot
- Box plot
- Area plot

Line plot

- Line plot can be created with DataFrame.plot() function.

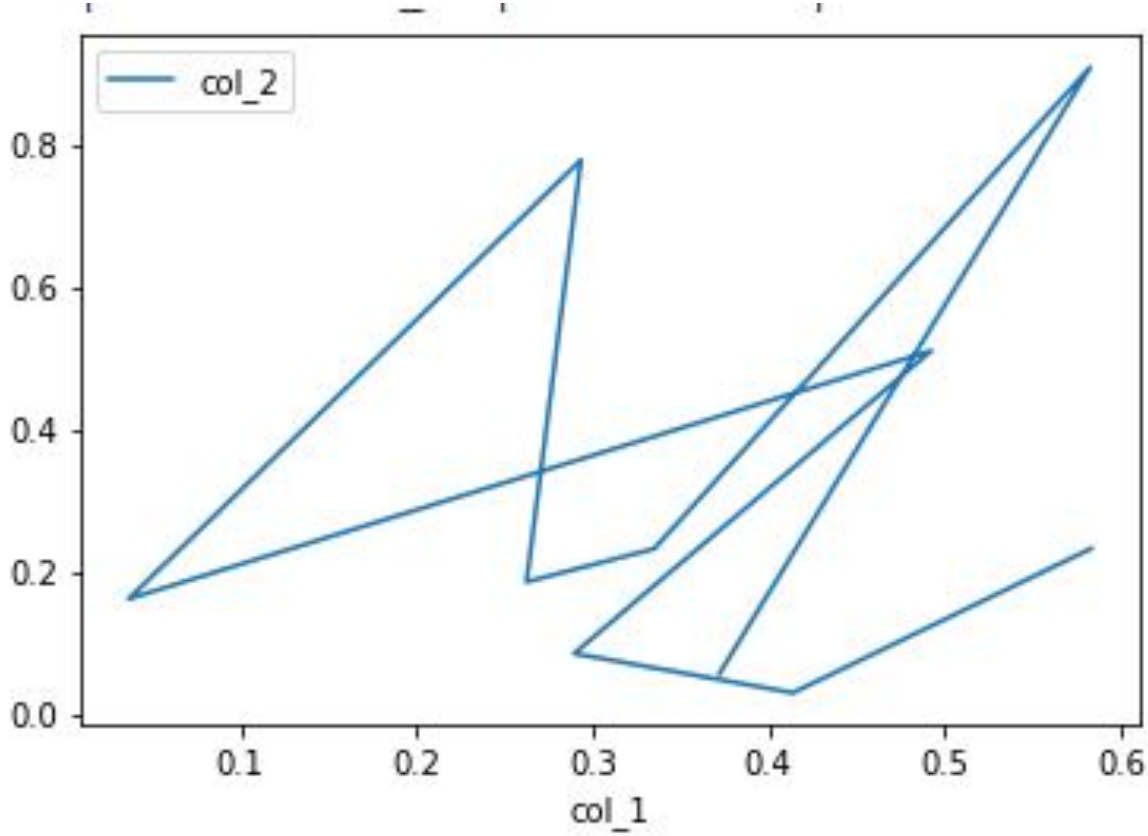
  df.plot()



Line plot

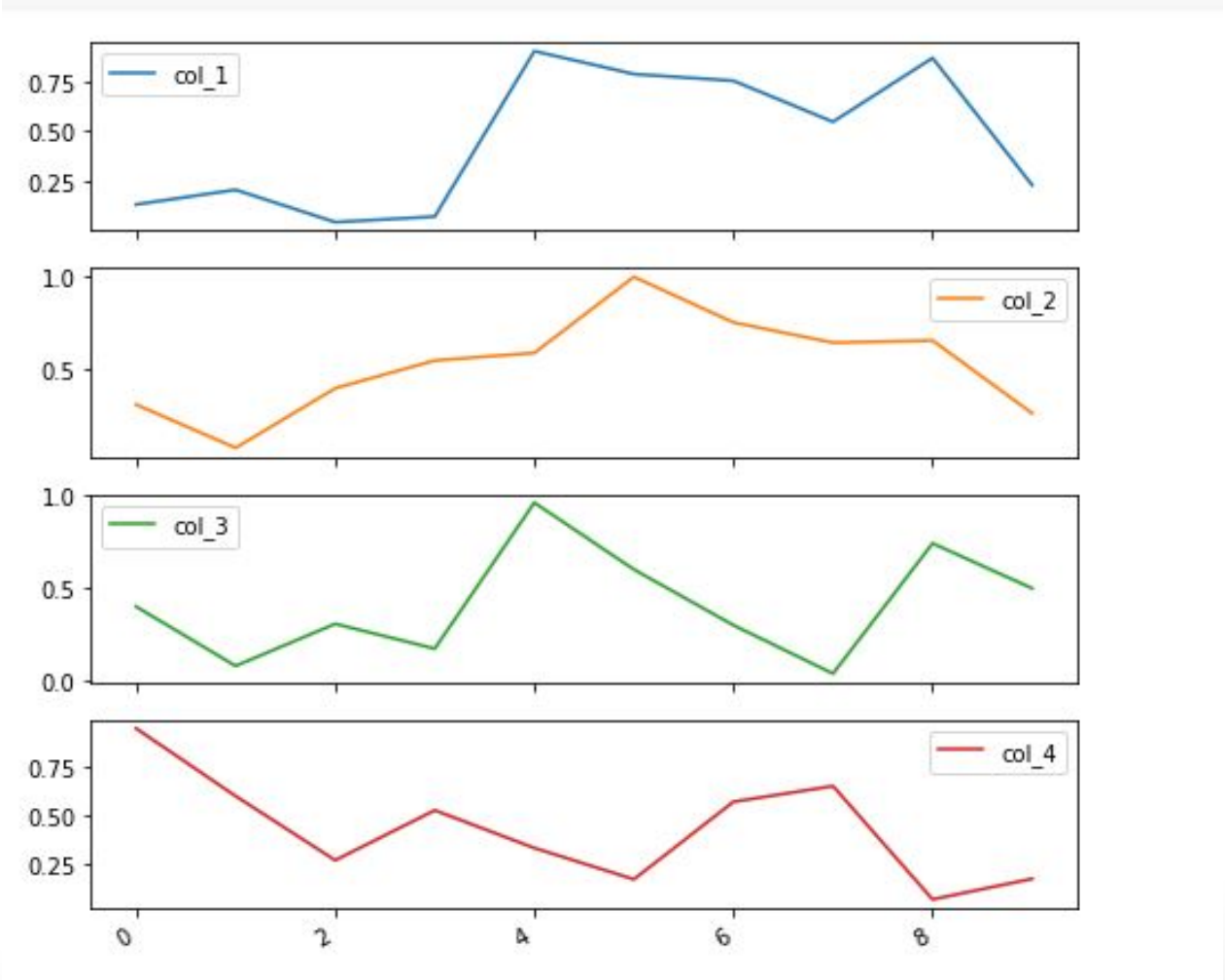
- Line plot can be created with DataFrame.plot() function.

  df.plot(x="col_1", y="col_2")



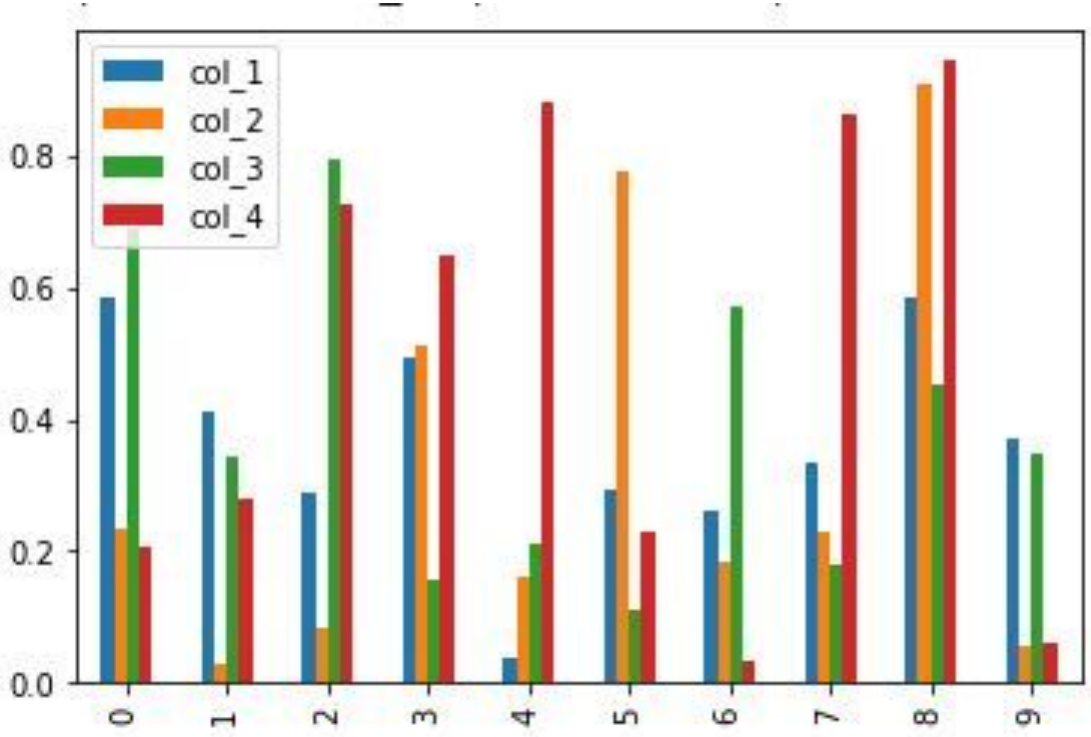
Line plot

- df.plot(subplots=True, figsize=(8, 8));



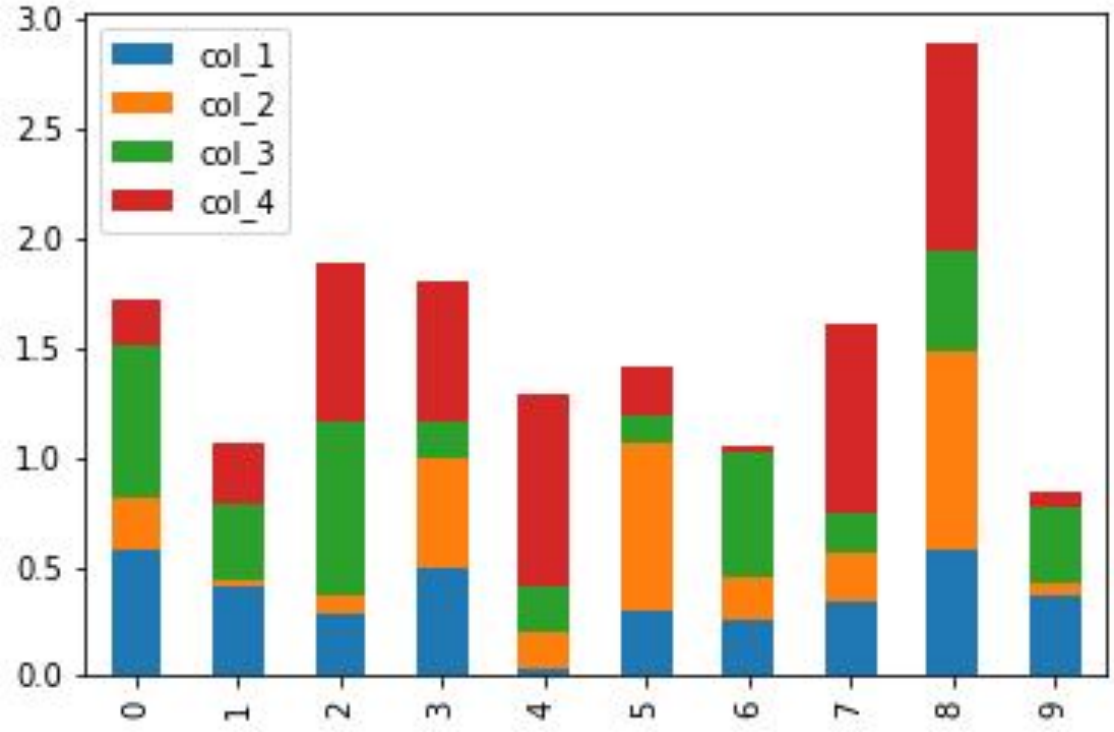
Bar plot

- Now, we will create bar plots for the same dataframe. Bar plot can be created with DataFrame.plot.bar() function.
- **df.plot(kind="bar")**



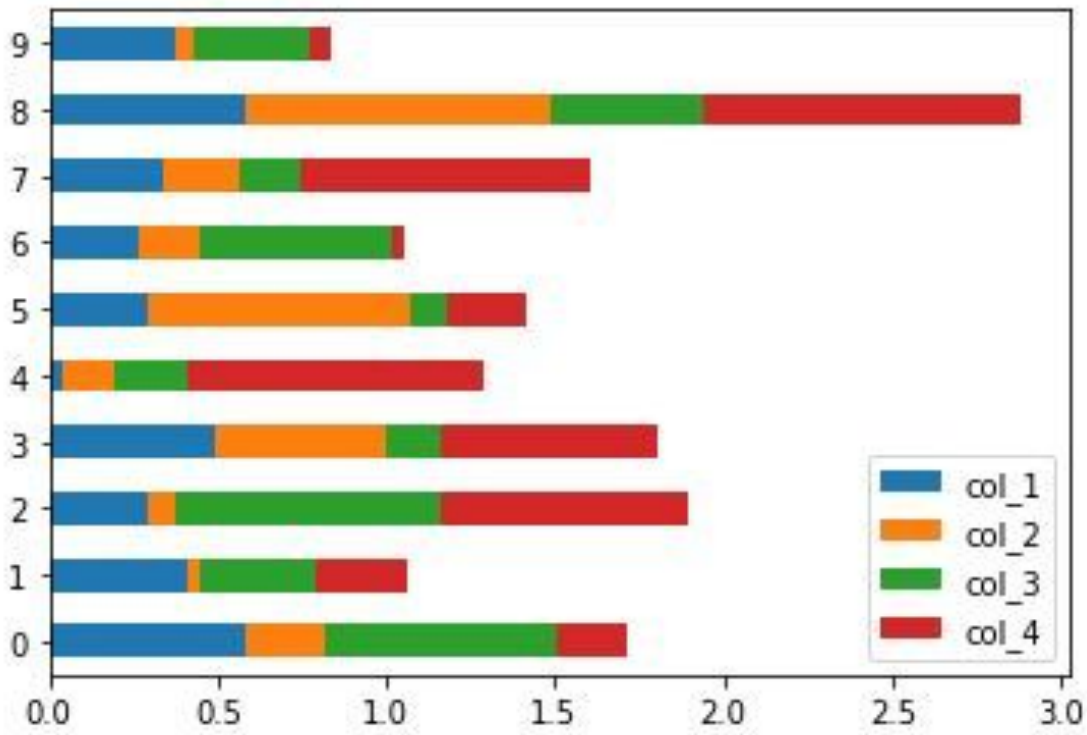
Bar plot

- **df.plot.bar(stacked=True);**



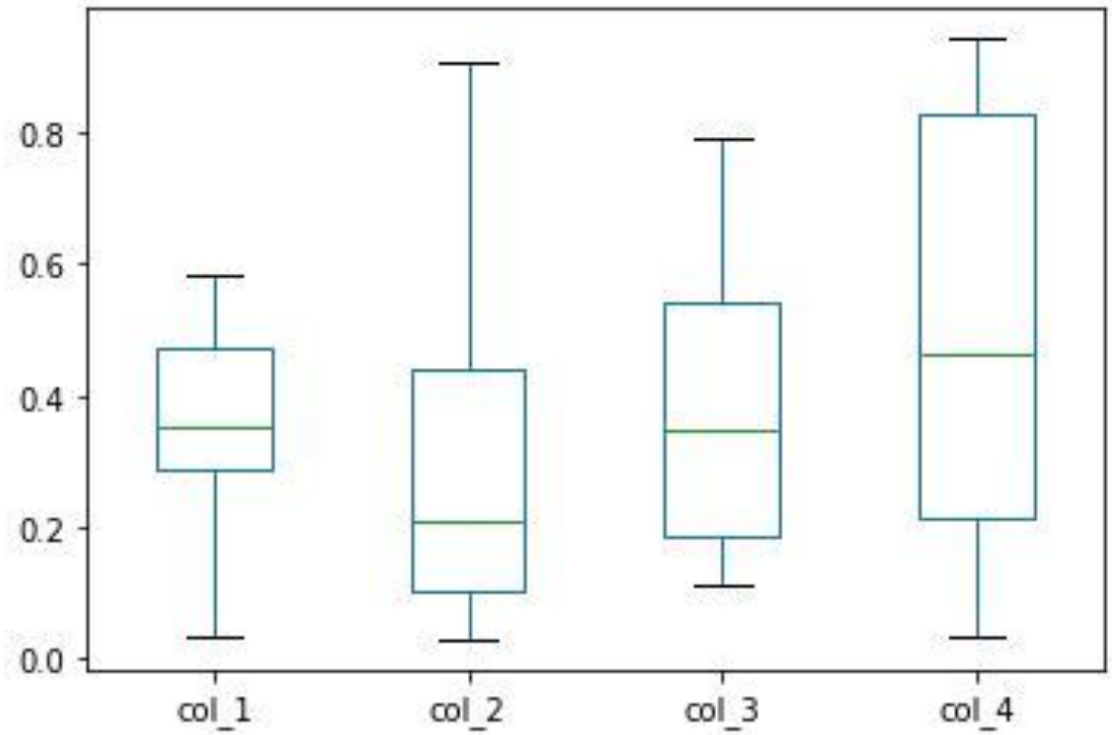
Bar plot

- **df.plot.barh(stacked=True);**



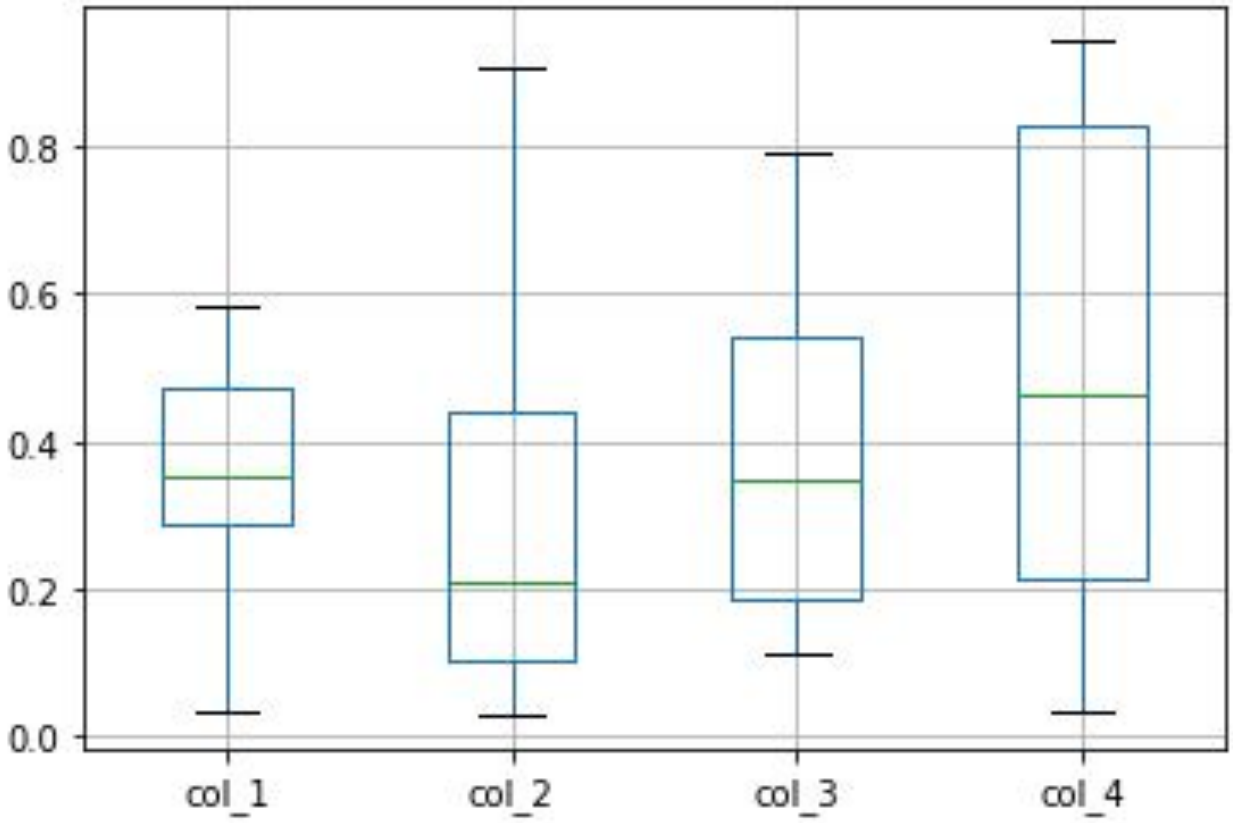
Box plot

- Box plot can be created with DataFrame.plot.box() function or DataFrame.boxplot().
- **df.plot.box()**



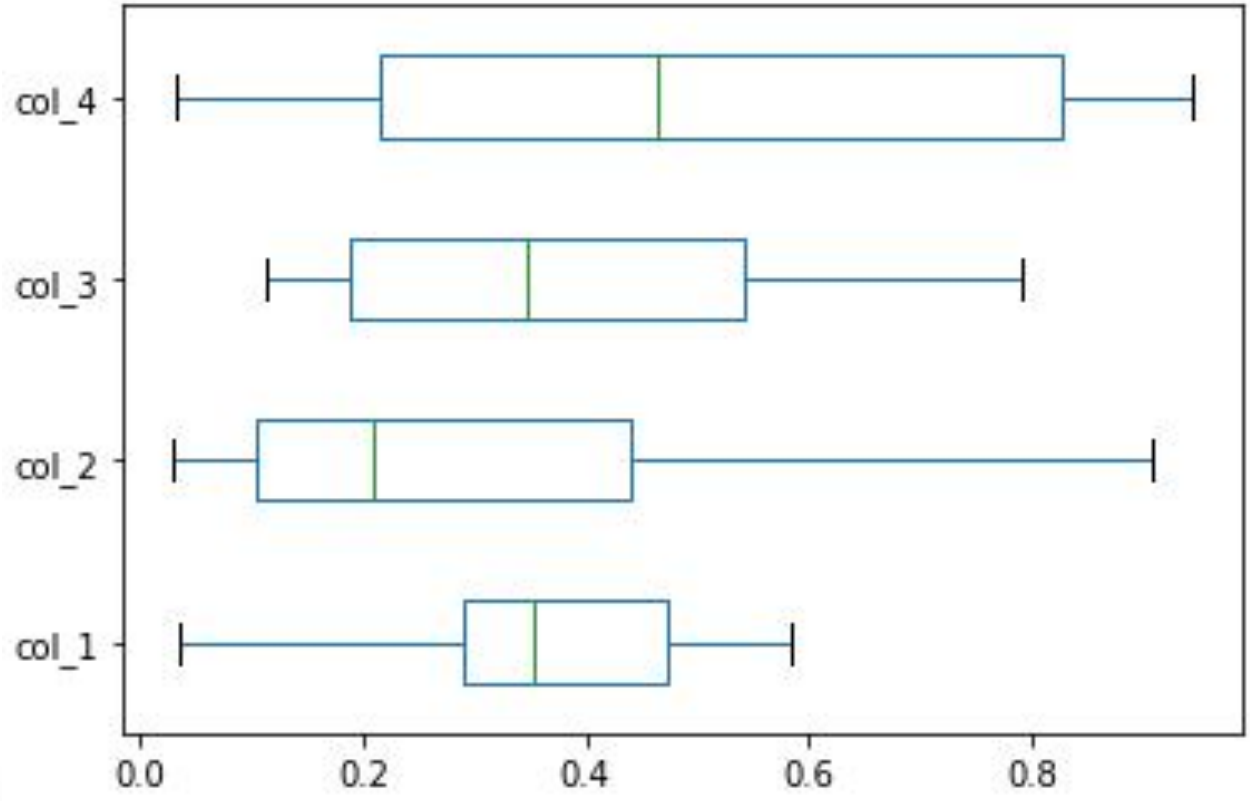
Box plot

- **df.boxplot()**



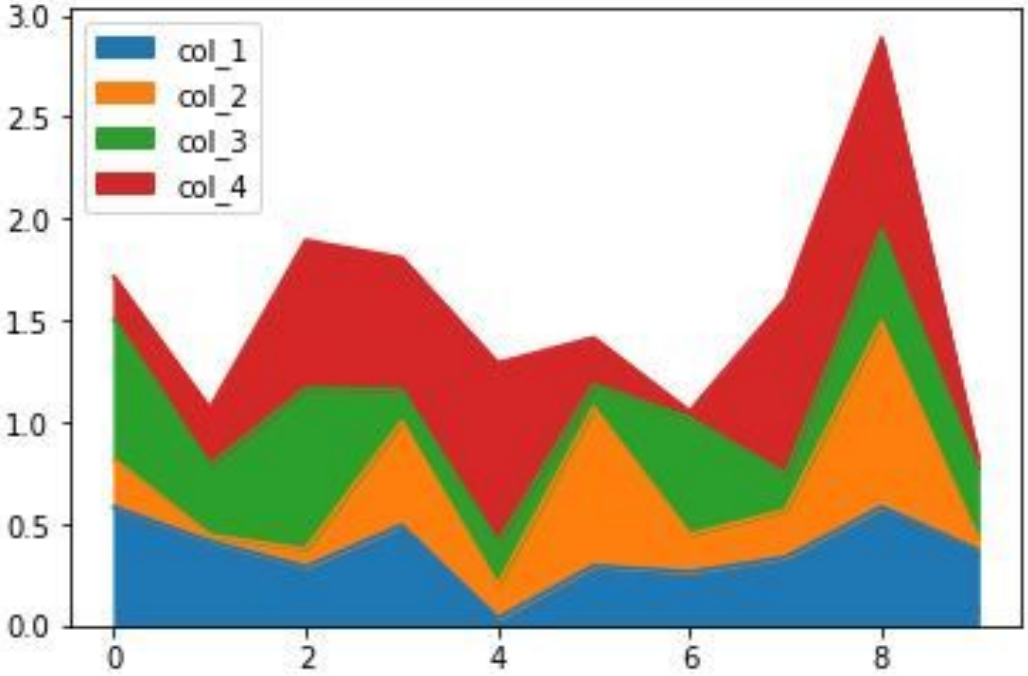
Box plot

- df.plot.box(vert=False, positions=[1, 2, 3, 4]);



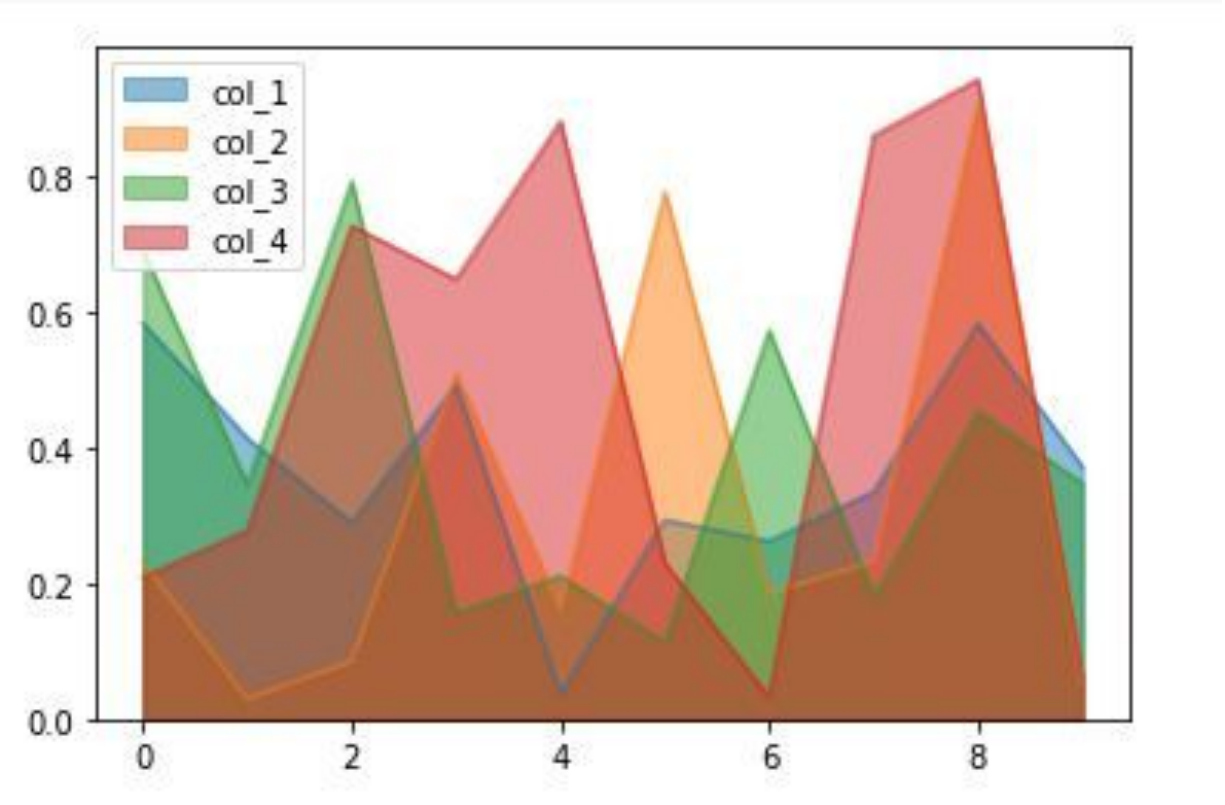
Area plot

- Area plot can be created with DataFrame.plot.area() function.
- **df.plot.area()**



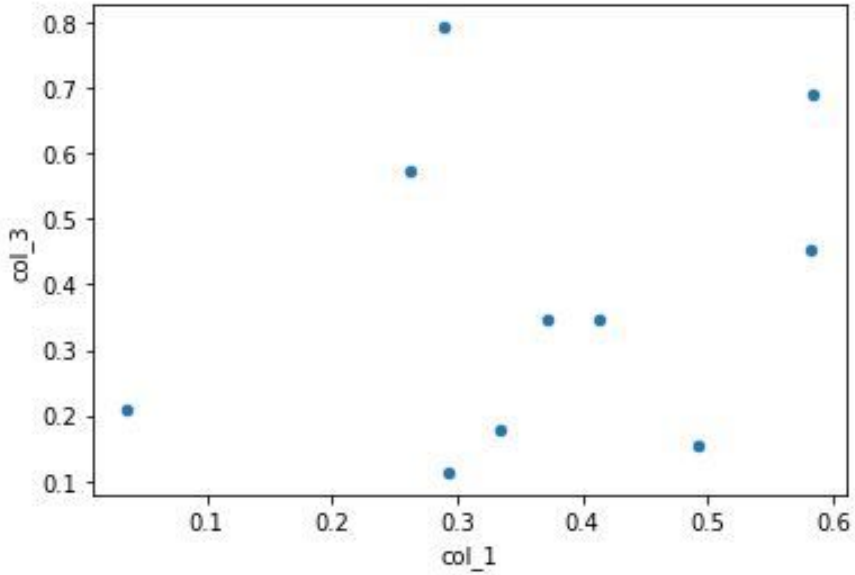
Area plot

- df.plot.area(stacked=False)

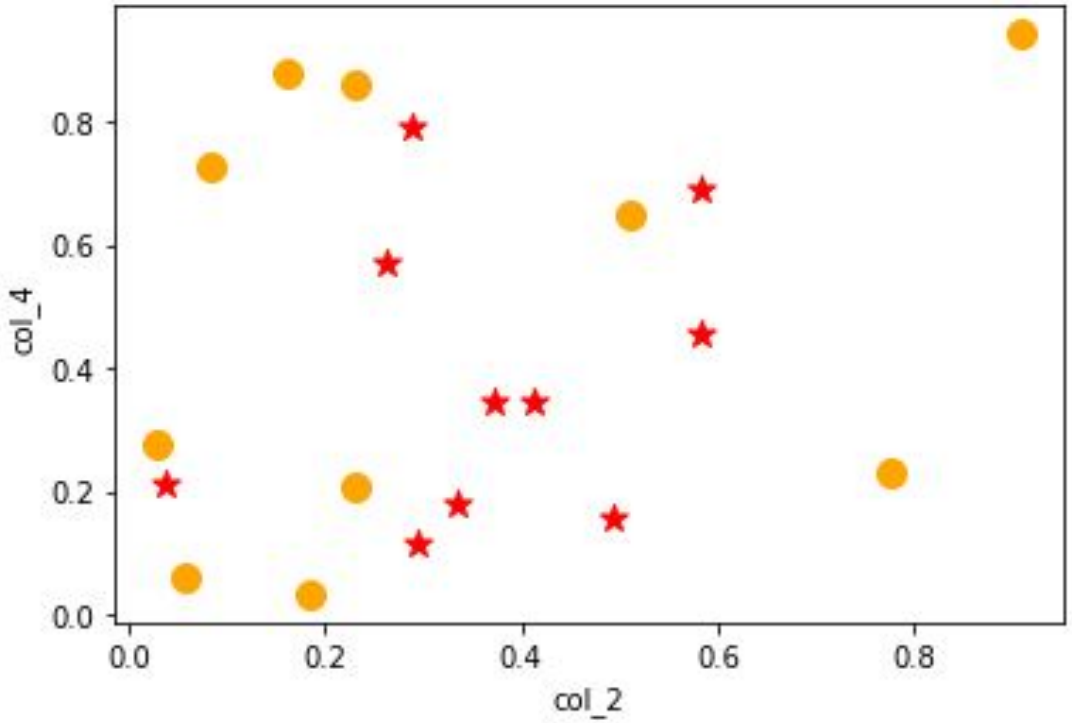


Scatter plot

- A Scatter plot can be created with DataFrame.plot.scatter() function. As we know scatter plot takes two-positional required arguments i.e. x and y to plot the graph. So, we will give the values of the x and y-axis as the name of columns.
- df.plot.scatter(x='col_1', y='col_3');

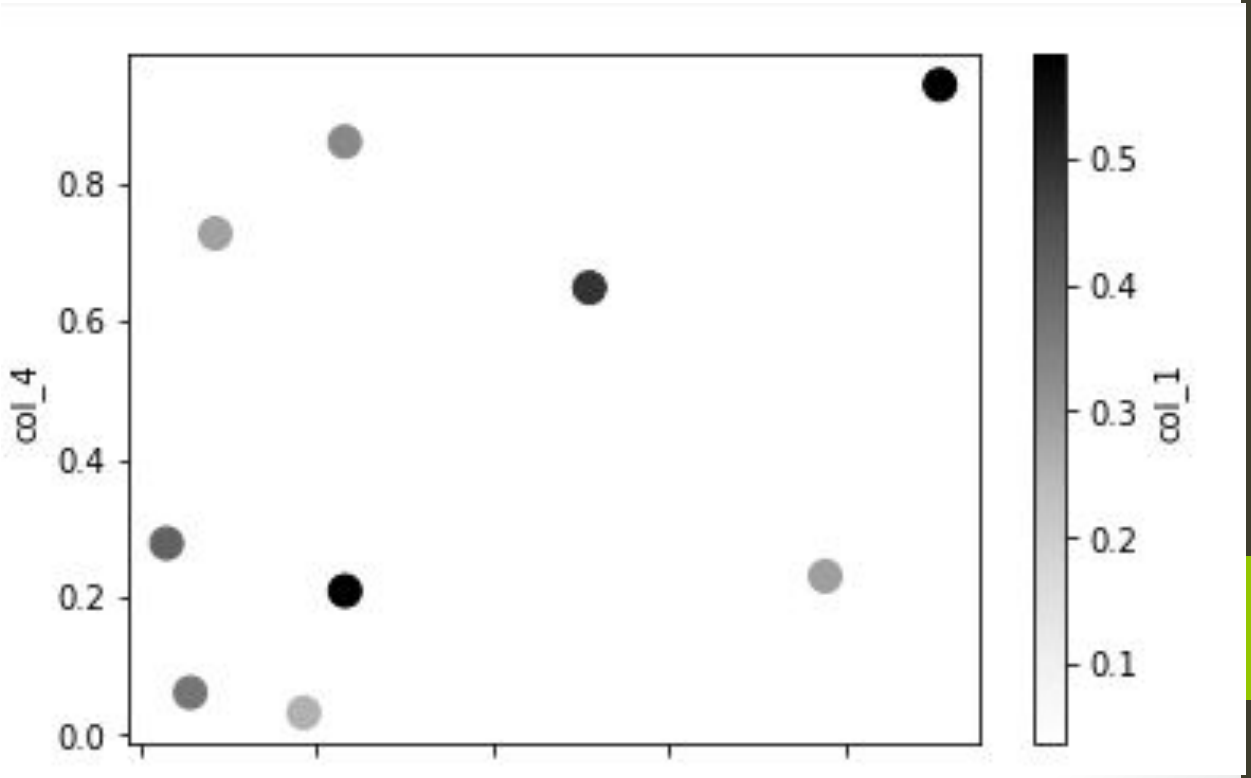


Scatter plot

- **ax=df.plot.scatter(x="col_1", y="col_3", color="red", marker="*", s=100)**
- df.plot.scatter(x="col_2", y="col_4", color="orange", s=100, ax=ax)
- 

Scatter plot

- df.plot.scatter(x="col_2", y="col_4", c='col_1', s=100)
- The c keyword is given as the name of a column to provide colours for each point.



Pie chart

- A Pie plot can be created with DataFrame.plot.pie() function or Series.plot.pie(). To generate a pie chart we will create series data as a pie chart is created only for one column.

pie = pd.Series(np.random.rand(5))

pie

```
0    0.067722
1    0.875775
2    0.239062
3    0.860075
4    0.595066
dtype: float64
```

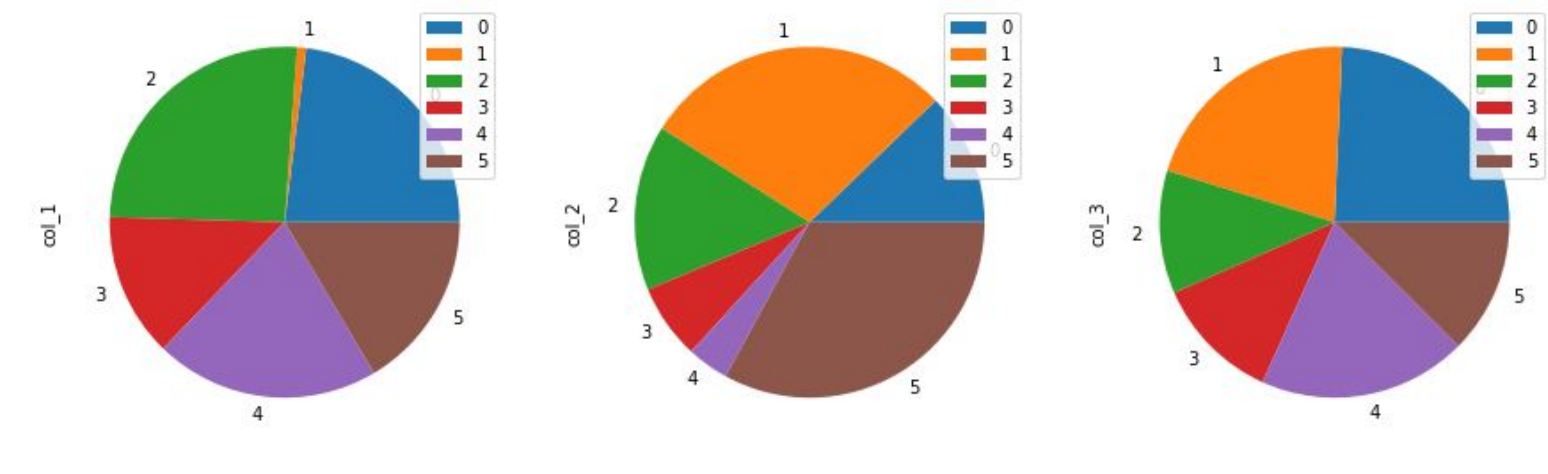
Pie chart

- #creating a DataFrame df2 = pd.DataFrame(np.random.rand(6, 3), columns=('col_1', 'col_2', 'col_3'))

	col_1	col_2	col_3
0	0.814209	0.367809	0.947977
1	0.029530	0.870160	0.816744
2	0.907597	0.460161	0.442202
3	0.469319	0.208575	0.454594
4	0.733588	0.116372	0.751799
5	0.582027	0.994945	0.486223

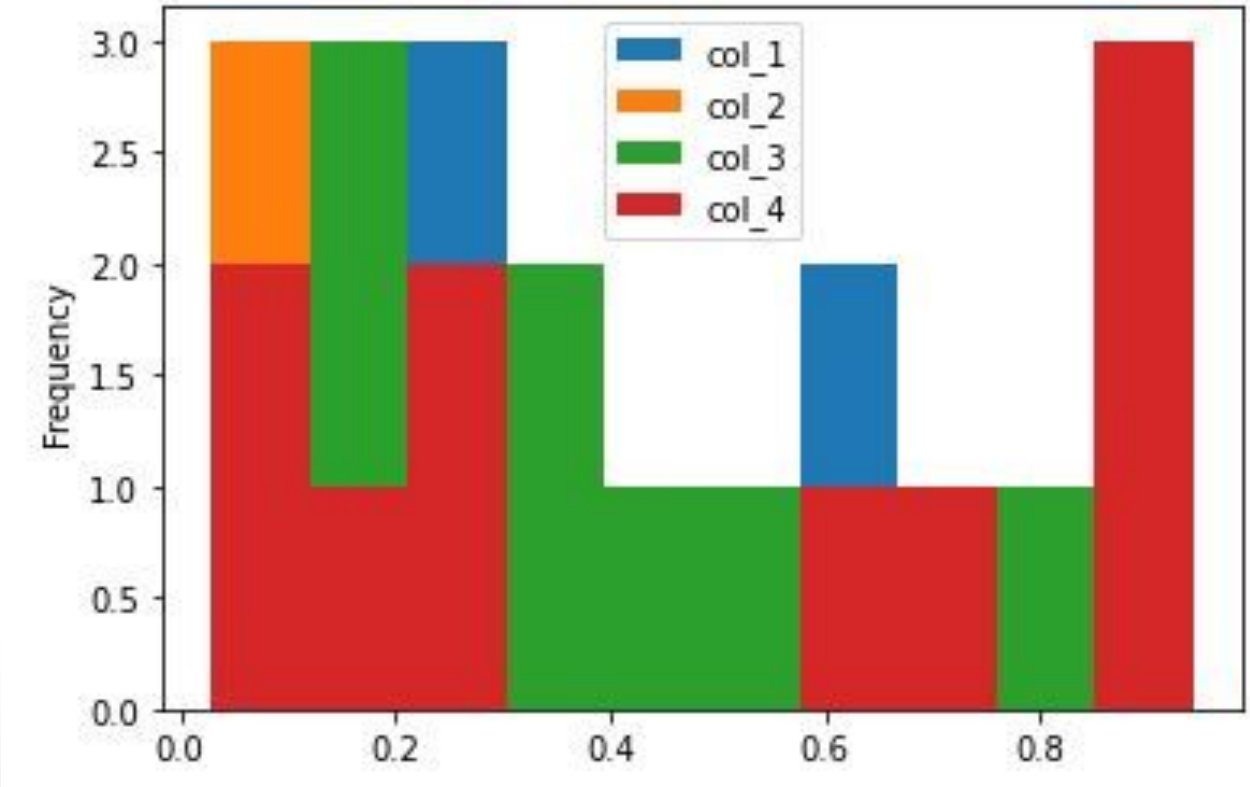
Pie chart

- df2.plot.pie(subplots=True, figsize=(15, 15))



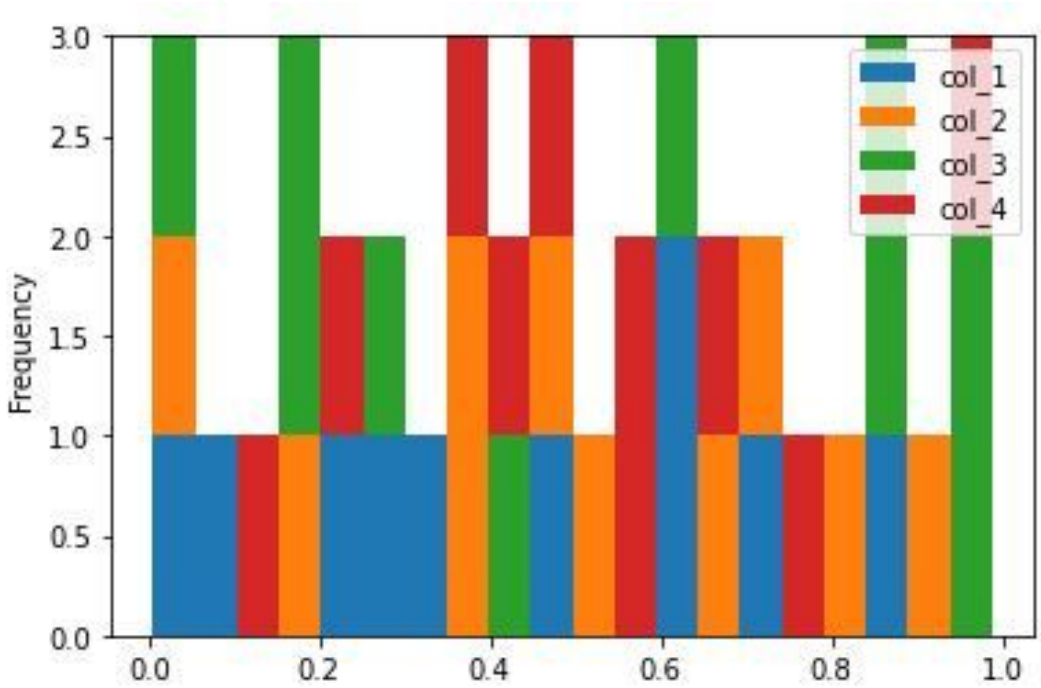
Histogram plot

- Histogram plot can be created with DataFrame.plot.hist() function.
- **df.plot.hist()**



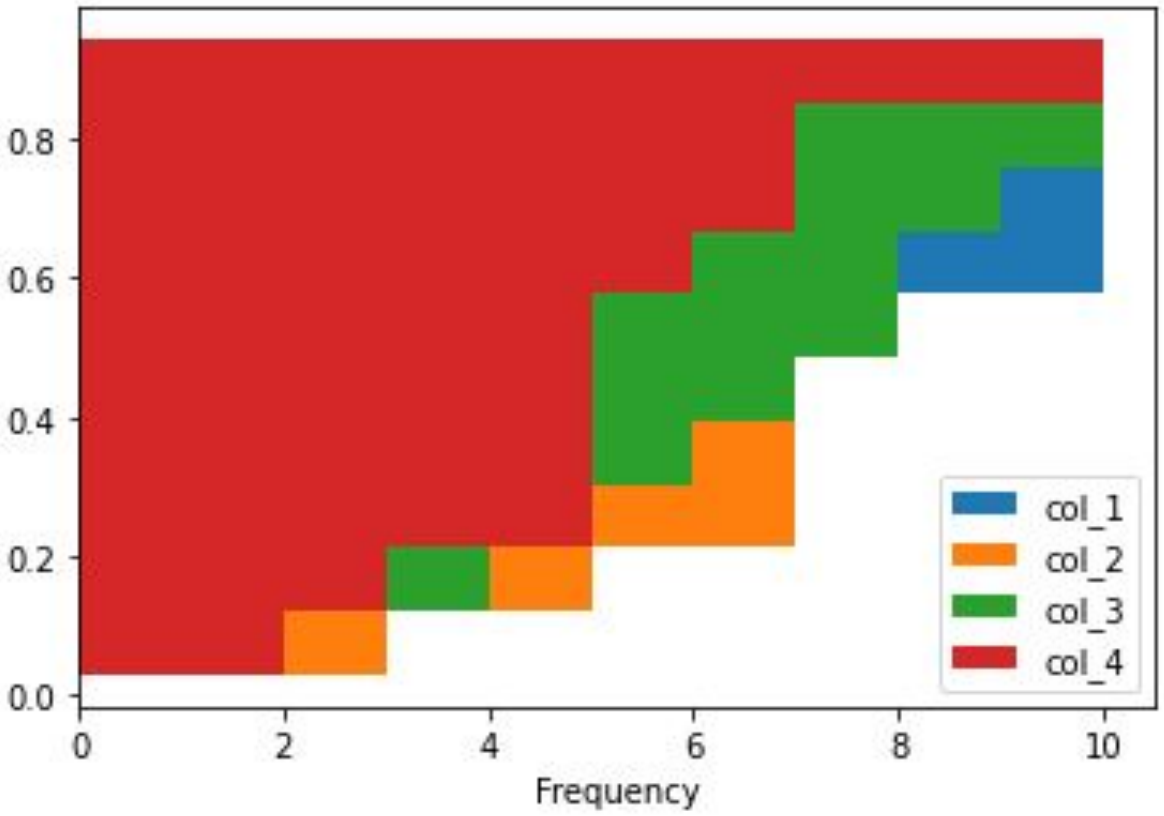
Histogram plot

- Histogram plot can be created with DataFrame.plot.hist() function.
- **df.plot.hist(stacked=True, bins=20);**
-



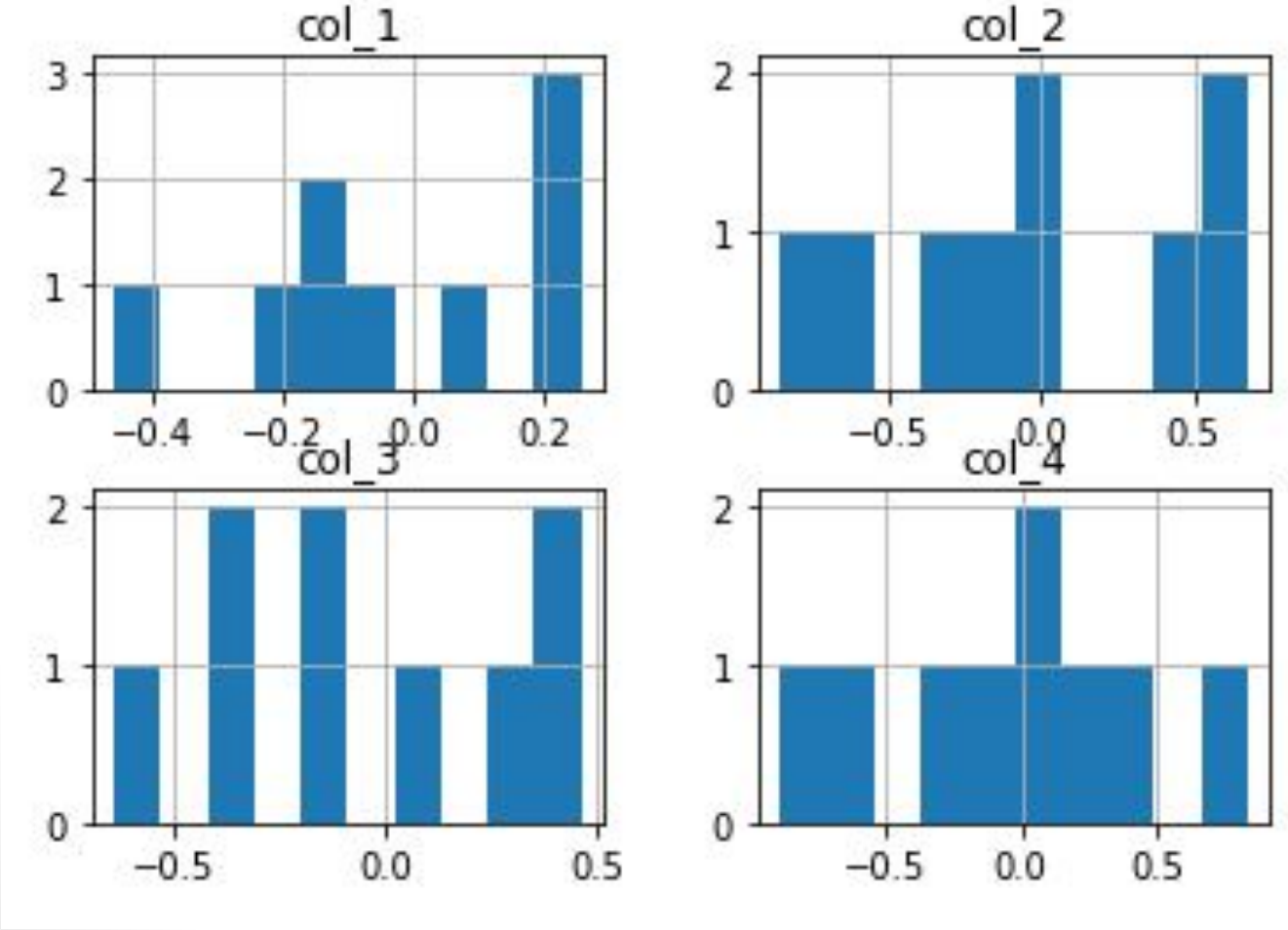
Histogram plot

- df.plot.hist(orientation="horizontal", cumulative=True);



Histogram – each column

- **df.diff().hist();**



Exploratory Data Analysis

- Exploratory Data Analysis is majorly performed using the following methods:
- **Univariate analysis:-** provides summary statistics for each field in the raw data set (or) summary only on one variable. *Ex*:- CDF,PDF,Box plot, Violin plot.(don't worry, will see below what each of them is)
- **Bivariate analysis:-** is performed to find the relationship between each variable in the dataset and the target variable of interest (or) using 2 variables and finding the relationship between them.*Ex*:-Box plot, Violin plot.
- **Multivariate analysis:-** is performed to understand interactions between different fields in the dataset (or) finding interactions between variables more than 2. *Ex*:- Pair plot and 3D scatter plot.